Project Report

By Alex (@a1e0xye) | 2026/3-2026/4

Introduction

Gomoku Game, also named as 5-in-a-row. With a typical board size of 15x15. Two players are involved in this game, where one of them using “X” or “O” as represent their dice on the board. When there are 5 same dice in a row (including vertical, horizontal or diagonal), the player wins the game. But if the cell in the board in filled, then no one wins the game as no one archived the win condition.

This program is coded using Python 3, which is a common program language nowadays. Under simple text-based interface, the game board is in the terminal, and players can make them decision by typing in their selected cell’s row and column number. The program will then check the row and column number inputted whether is valid or not, or even the cell selected is occupied or not. Finally, player’s dice will be placed on the board and switched to other players turn to continue.

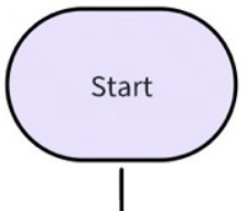
In this project report, more detailed will be explained. Enjoy???

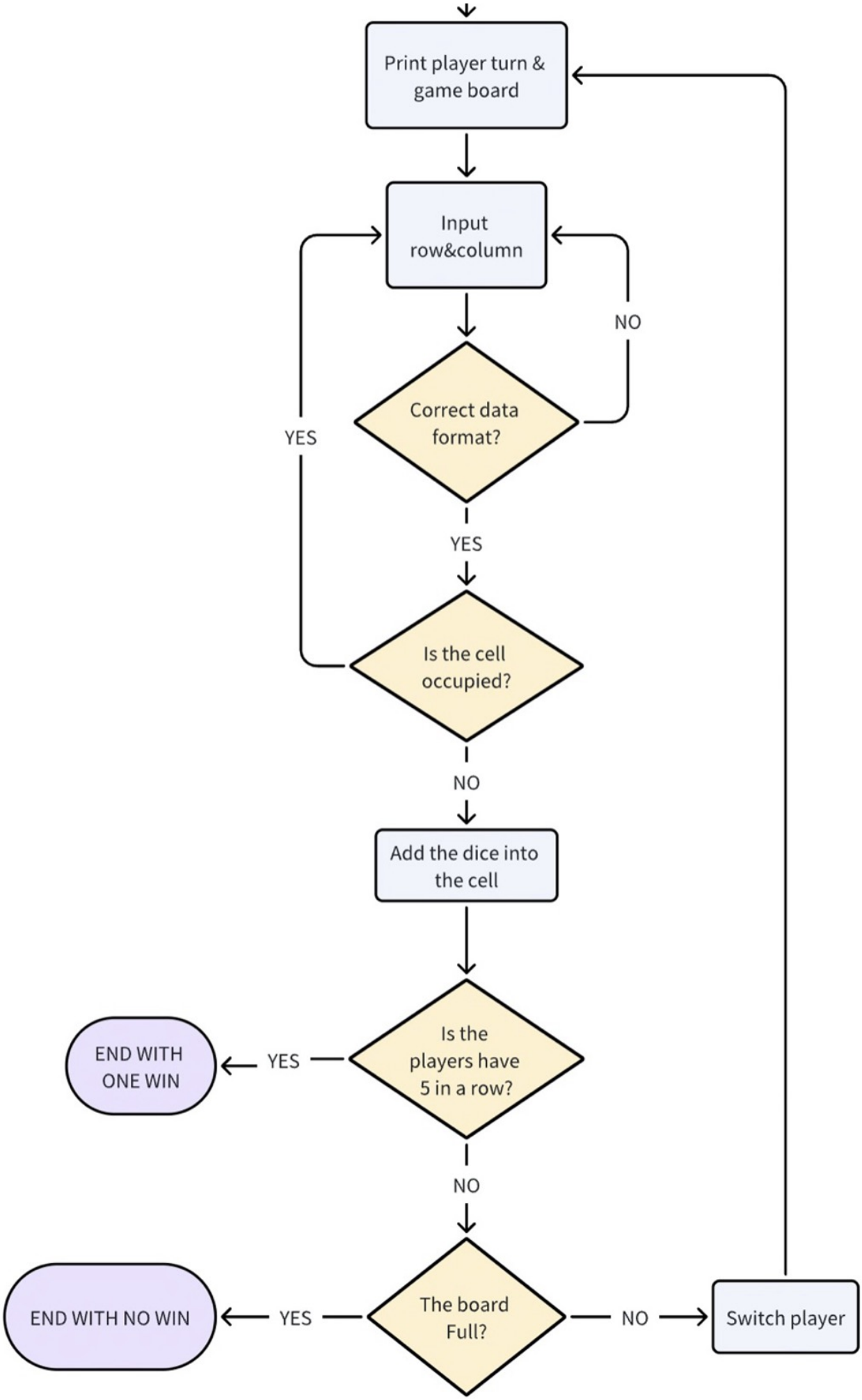
Game features

- Game Board: Represent the Gomoku board as a grid of a size typically 15x15, where each cell can be either empty, occupied by 'X', or occupied by 'O'.
- Player Turns: Implement the game logic to alternate between the 'X' and 'O' players, allowing each player to place a stone by selecting an empty cell on the board.
- Win Conditions: Check for a winning condition after each move, where a player wins if they align five of their symbols (X or O) consecutively in a row, column, or diagonal.
- Draw Condition: Detect when the game reaches a stalemate, if all cells on the board are filled and neither player has achieved a winning condition.
- User Interface: Provide a simple text-based user interface for players to interact with the game, which includes displaying the current board state and allowing players to enter their moves.
- Input Validation: Ensure that the user's input is valid by checking if the selected cell is empty and falls within the valid range of the board.
- Game Loop: Implement a game loop that continues until a winner is declared or the game ends in a stalemate.

Program logic flow overview

The following flow chart illustrates the logic flow of the game. Program are coded based on this logic flow.




Print player turn & game board
Input row&column
NO
Correct data format?
YES
YES
Is the cell occupied?
NO
Add the dice into the cell
Is the players have 5 in a row?
YES
END WITH ONE WIN
NO
The board Full?
YES
END WITH NO WIN
NO
Switch player

Variable/constant declaration and initialization

Name	Type	Initial value	Notes
`turn`	variable	0 [Ln 15]	Recorded which player's turn currently. The value will be 0 or 1, as the represent of player 1 or 2
`empty_cell_indicator`	constant	(Custom value) [Ln 16]	When the cell on the board is empty, the board will show this value as it indicate a empty cell.
`player_1_indicator`	constant	(Custom value) [Ln 17]	The indicator for player 1. It will show on the board cell if the player selected the cell. It should be different from player 2.
`player_2_indicator`	constant	(Custom value) [Ln 18]	The indicator for player 2. It will show on the board cell if the player selected the cell. It should be different from player 1.
`board`	2D List	Filled with `empty_cell_indicator` [Ln 20-25]	Stored all cell value.
`num_row_column`	constant	(Custom value) [Ln 21]	The value determine the size of the board (if num_row_column = 15, then the board will have 15 rows and columns). The board size can be easily changed by change the value of this varibale.

Initialization of 2D array

```
board = []
num_row_column = 15 # Board Size
for i in range(num_row_column):
    board.append([])
    for r in range(num_row_column):
        board[i].append(empty_cell_indicator)
```

The for-loop add the `empty_cell_indicator` into all cell of the board, with the size of board. It will append row by row, and the value of `num_row_column` determine how many row and column will be added.

External Library

> **Note**: In this project, external libraries do not directly affect the main logic; they only serve to improve the game experience. Even if they are removed (the relevant functions are replaced), the project will still run according to the expected logical flow.

Library can help us archive specific function, it can (make the code easy simple, strc more clear). Library can be easily install to computer by `pip install`, and `import` to the code.

In this project, the following library is used.

tabulate

> https://github.com/astanin/python-tabulate

`tabulate` is a python3 library that help output a formatted tabular data, provide a clear table. In this project, we make the game baord into a 2D array, by using this library, we can print the data in the 2D array in a clear way.

Following function provided by the library is used

Function Name	Used line
tabulate	Ln 41

`tabulate()`

This function takes a list of lists or another tabular data type as the first argument, and outputs a nicely formatted plain-text table.

It can be used by `tabulate(table,headers,tablefmt,stralign,numalign)`.

The table format can be customize by changing the value in `tablefmt`,`stralign`, `numalign`.

termcolor

> https://github.com/termcolor/termcolor

`termcolor` can help us output in terminal with color, instead of plain text with no color. It will defintely enhance the user experience compared to original, especially for terminals in white & black.

Following function provided by the library is used

Function Name	Used Line
cprint	Ln 94,95,97,98,106,116,127,129,136,142,149

`cprint()`

This function output the message with text colors, text highlights.

It can be used by `cprint(message,textcolors,texthighlights)`

Example: `cprint("Hello, World!", "green", "on_red")`

If the library is imported unsuccessfully, the program will occur a (run-time error). So at the start of the code, a `try-except` is used.

```
try:
    from tabulate import tabulate
    from termcolor import colored, cprint
except ModuleNotFoundError as e:
    raise ModuleNotFoundError(
        "Missing dependency. Please follow instructions from README.md"
    ) from e
```

The `try` block lets us test a block of code for errors. When there is error, the `except` block lets us handle the error. In this project, this help us to show a notice message to user when the program unsuccessfully import library.

Modular approach

The program added different function, as specific function will be used frequntly. To define they as a function, the main loop can be more simple, as it reduce repeated code in it.

By using `def`, we could define our module function. Let `def func(number)` be a example, `func` is the function name, `number` is the value that passed into the function.

In this project, The following logic will be defined as functions to achieve reuse.

`board_print(board)`

This function is to print the gomoku board to terminal.

```
# Ln 29-49
def board_print(board):
    headers = [" "]
    for i in range(1, num_row_column + 1):
        headers.append(str(i))

    rows = []
    for i in range(num_row_column):
        row = [str(i + 1)]
        for cell in board[i]:
            row.append(cell)
        rows.append(row)
    print(
        tabulate(
            rows,
```

```
            headers=headers,
            tablefmt="rounded_grid",
            stralign="center",
            numalign="center",
        )
    )
    print()
```

The function will first create an empty array `headers`, and then use a `for-loop` to add row/column number into it. And then `rows` is created as a 2D array for all rows, the row number will be added at the first of each row.

The 2D array `rows` will be printed out as a formatted table by using `tabulate`, after the row and column number has been added into array `rows`. As mentioned in the previous part, the format of the table and text align can be changed by replacing the value of `tablefmt`,`stralign`,`numalign`.

`check_win_v2(board, row, col, indicator)`

This function is to check for a winning condition after each player's move, where a player wins if they align five of their symbols consecutively in a row, column, or diagonal (According to the game features).

```
# Ln 52-79
def check_win_v2(board, row, col, indicator):
    directions = [
        (0, 1),   # Horizontal
        (1, 0),   # Vertical
        (1, 1),   # Right-down diagonal
        (1, -1)   # Left-down diagonal
    ]
    for dr, dc in directions:
        count = 1  # Initial count

        # Positive
        r, c = row + dr, col + dc
        while 0 <= r < len(board) and 0 <= c < len(board[0]) and board[r]
[c] == indicator:
            count += 1
            r += dr
            c += dc
        # Negative
        r, c = row - dr, col - dc
        while 0 <= r < len(board) and 0 <= c < len(board[0]) and board[r]
[c] == indicator:
            count += 1
            r -= dr
            c -= dc

        # Satisfy win condition: 5 in a row (or more)
        if count >= 5:
            return True
```

```
    return False
```

`direction` stores the increase/decrease step for both the row and the column when the program checks for a winning line. In the list `directions = [(0, 1), (1, 0), (1, 1), (1, -1)]`, each pair `(dr, dc)` represents one possible direction on the board:

- `(0, 1)` : Move horizontally to the right (ROW: No change ; COLUMN: +1 each step)
- `(1, 0)` : Move vertically downwards (ROW: +1 each step ; COLUMN: No change)
- `(1, 1)` : Move diagonally down-right (ROW: +1 ; COLUMN: +1 each step)
- `(1, -1)` : Move diagonally down-left (ROW: +1 each step ; COLUMN: -1 each step)

For each direction, the function first moves to the "positive" direction (adding `dr` and `dc` to the current position) and counts how many same pieces are connected. Then it moves to the "negative" direction (subtracting `dr` and `dc` from the current position) to count the connected pieces on the opposite side. By using these `direction` steps, the function can scan along a straight line in all four possible directions and count same pieces number in a row starting from the latest move and then decide whether the player have met the win condtion or not.

`is_board_full(board)`

This function is to check the cells in game board is all filled or not.

```
# Ln 82-86
def is_board_full(board):
    for row in board:
        if empty_cell_indicator in row:
            return False
    return True
```

It use for-loop to to find empty cell in a row, if found, it mean the game board are not all filled yet, so a boolean value "False" will be return (Meaning that the board are not full). If it finished searching for empty cell in row, but still cannot found, a "True" will be return to tell that the game board is full, and the game reaches a stalemate.

Scope of Variables and Parameters Passing

Global scope

Global variables are available from within any scope, global and local.

Global variables in this program are listed below:

Variable	Type of Scope	Purpose
`turn`	Global	Stores whose turn it is (`0` for Player 1, `1` for Player 2)

Variable	Type of Scope	Purpose
`empty_cell_indicator`	Global	Symbol used to represent an empty cell on the game board
`player_1_indicator`	Global	Symbol used by Player 1
`player_2_indicator`	Global	Symbol used by Player 2
`board`	Global	2D array that stores all game board cell and moves
`num_row_column`	Global	Board size

Local scope

A variable created inside a function belongs to the local scope of that function, and can only be used inside that function.

Local variables in this program are listed below:

Function	Local Variable(s)	Purpose
`check_win_v2(board, row, col, indicator)`	`directions`, `dr`, `dc`, `count`, `r`, `c`	Used temporarily to scan all four directions and count number of same symbol to check the win condition
`board_print(board)`	`headers`, `rows`, `i`, `row`, `cell`	Used temporarily to print the game board with formatted table by using tabulate
`is_board_full(board)`	`row`,`board`	Used to traverse all cell in the game board to find empty cell.

Local variables can help each function stay independent, reduces accidental variable conflicts with main, and improves readability and maintainability. A local variable in a function can also use `gloabl` to make the variable global.

Parameter Passing

Parameter passing means sending values into a function when excuting the function so it can use those values inside the function. In this program, functions receive data such as the game board, player selected cell position, and player indicator through parameters. The function wont affect the original value outside the module. This makes the functions reusable, same function can work with differnt inputted with the same processing logic, and without declaring variables again.

Parameter passing in this program are listed below:

Function	Parameters	Notes
`board_print(board)`	`board`	Passes the current game board so the function can display the latest game board in a formatted table to players.

Function	Parameters	Notes
`check_win_v2(board, row, col, indicator)`	`board`, `row`, `col`, `indicator`	Passes all data needed (Current game board, the cell where player chosen at this round, the indicator for the player) to check whether the player met the win condition (5-in-a-row in any direction)
`is_board_full(board)`	`board`	Passes the current game board so to check the board is filled or not.

Parameter passing in this project improves modular design, it ake each function has their own specific responsibility and only receives necessary data without affecting variables outside it.

Interface of the Program

This project is a **text-based interface**. The user only need to run `main.py` (Specific librarys are required to install in advance). This project do not offer graphic windows, all text input/output are printed in terminal with keyboard input, no mouse clicking required.

Runtime dependencies

The program required two libraries to run

- `termcolor` and `tabulate`

If these library are not installed, the program will raise a `ModuleNotFoundError` and print a message to tell the user to follow instructions to run this code.

Input interface

Player need to enter their chosen cell's row and column number in **two separate lines** of input. Example be like:

```
Enter the row:
Enter the column:
```

This two message are printed in blue and bold, to better distinguish between program output and user input.

Coordinates of cell are in number, including both row and column. As `input()` cannot limit what type of data can user type in. The expected input data type is digits only(within the board size). Empty input(Submit without typing anything in), non-numeric input, value that exceeded the board size, or chosen an occupied cell should all be labelled as invalid input. Therefore, a while loop was used to allow the user to keep retrying input until a valid value was entered.

Output interface

By using ANSI Escape code (`\033c`), the terminal can be cleaned in each turn,so the player wont see the old game board and messages that affect the gaming experience.

The interface show these element:

Output element	Description
Turn banner	“Player 1 turn” (show in green) or “Player 2 turn” (show in red), followed by the piece symbol (from `player_1_indicator` or `player_2_indicator`) that player uses.
Game Board	Game board printed by using formmated table library `tabulate`, with row and column number at top and left side. Empty cells appear as a space (default, but actually can be changed by replacing the value in `empty_cell_indicator`); Occupied cells show the player's symbol who selected this cell.
Error messages	(Appear when error occur) Messages are show in red bold colour when the inputted value are not in expected format (Empty, invalid, out of range, or the cell is occupied).
End states	When a player met the win condition or the game board are filled, the terminal will clean and show the final game board. A bold message will announces the result (“Player 1 wins!"/“Player 2 wins!”, or “Board Full. No winner.”).

Data Collection, Input and Validation

Data collection in this project is to read the player’s move from the keyboard in each turn. **Validation** is to check the input is valid and safe to continue that wont occur a run-time error. As `input()` return a string value, the program must check it is in digits only, if not, the program will return a error when excuting `int()`. We cannot guarantee that player only type in what the program want, they may type in a whitespace, a chinese name, a "@", etc.. So, it is important to do validation.

Step 1 — Read raw input

Each move uses two `input()`: `Enter the row:` and `Enter the column:`. The program will receive two strings. In this step, the program only **collects** data, without checking the data.

```
# Ln 104-105
selected_row = input(colored("Enter the row: ", "blue", attrs=["bold"]))
selected_column = input(colored("Enter the column: ", "blue", attrs=
["bold"]))
if selected_row == '' or selected_column == '':   # Input nothing
    cprint("You type nothing!!! Try again, please.", "red", attrs=
["bold"])
    continue   # Jump to next loop: Input again
selected_row = selected_row.strip() # Remove the whitespace
selected_column = selected_column.strip()
```

Item	Detail

Item	Detail
Source	`input()`
Data type	`str` (Always)
Special case	If one of the returned string is empty (`''`)(The player submit without type in anything) **before** doing the `strip()`, the program return the error message ("You type nothing") and ask the player to input again.
After collecting	By using`strip()`, the leading/trailing whitespace in value stored in `selcted_row`and `selected_column` will be removed. >>> **Example**: `" 3 "` will be changed to `"3"`, so it become a digit only value, safe to continue processing.

Step 2 — Format and type checks

After stripping, the program requires both strings to be **digits only** (`isdigit()` help us to verify). If that passes, both values will be converted to `'int'` variables. If that not passed, meaning the strings contain non-digit, a error message ("invalid input") will be printed and the player will be asked to enter again.

```
# Ln 111-117
if selected_column.isdigit() and selected_row.isdigit():  # Check the
strings only contain digit number
    selected_column = int(selected_column)
    selected_row = int(selected_row)
    break
else:
    cprint("Invalid input", "red", attrs=["bold"])
    continue
```

Validation rule	Actions
Both strings are digit only(`isdigit()` returned `True`)	Both values will convert to `int` type and the inner input loop can exit.
Otherwise	Print error message “Invalid input”. Then, let the player to enter tha number of row and column again.

Step 3 — Range and board rules

After converting to two `int` variables, the program checks they are within the board size (From **1** to `num_row_column`), if not, a error message ("Invalid input") will be printed and let the player to input agian. If the value is within the board size, it then check the player selected cell on the game board, only if the cell is empty, the program place the current player’s piece at that cell. If the cell is occupied, the program will tell the player the cell selected is already occupied and allow the player to choose another one.

```
# Ln 119-129
if selected_row > 0 and selected_row <= num_row_column and selected_column
> 0 and selected_column <= num_row_column:  # Data validation - within the
```

```
board size
    if board[selected_row - 1][selected_column - 1] ==
empty_cell_indicator:   # Data validation - EMPTY CELL
        if turn == 0:
            board[selected_row - 1][selected_column - 1] =
player_1_indicator
        else:
            board[selected_row - 1][selected_column - 1] =
player_2_indicator
        break  # Exit inner loop - input valid & finish replacing the cell
    else:
        cprint("This cell is already occupied", "red", attrs=["bold"]) #
Repeat loop - Input valid & Cell occupied
else:
    cprint("Invalid input", "red", attrs=["bold"]) # Repeat loop - Input
invalid
```

Validation rule	Notes
`1 <= row <= num_row_column`	Make sure the selected cell is in the baord (Within board size)
`board[selected_row-1][selected_column-1] == empty_cell_indicator`	Prevent overwriting another player chosen cell.

Together, the nested `while True` can let player **"repeat input until valid"** : "Unlimited chance to input."

Data Processing

After the inputted data passed validation, the program will then **process** the move.

The main stages each turn are listed below.

Stages	Notes
Place move	[Ln 122, 124] Write `player_1_indicator` or `player_2_indicator` into the chosen cell in `board`.
Win check	[Ln 133, 139] Calls `check_win_v2(board, row_index, col_index, indicator)` to check if the move created five or more in a line in any of four directions (horizontal, vertical, two diagonals)
Draw check	[Ln 146] If in stage "Win check" returned the player have'nt win yet, then call `is_board_full(board)` to check if any empty cell remains in the game board. If no remaining empty cell, the game reaches a stalemate.
Turn switch	If the game continues (No one win & The board are not all filled), flips `turn` with `turn = 1 - turn` so the other player moves next (Alternates 0 and 1)

Data structures: Cells in the game board are stored in a **2D list** (list of rows, each row a list of cell values). Row/column indices used with `board` are start from 0, while user input is start from 1, so the program have

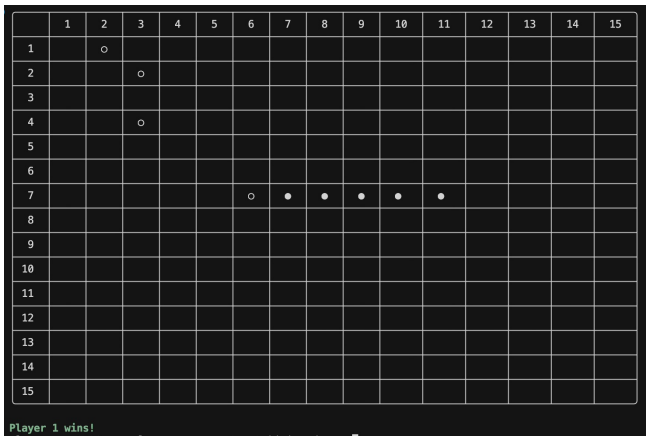
to subtract 1 when indexing.

Program Output

Output is everything that players can only **sees** in the terminal when playing this game. Output should be clear and simple, so that players can understant how to play&continue the game. In this project, all outputs are produced by `print`, `tabulate`, `cprint`, and also prompts in `input`.

Stage	Output contents
Start of each turn	Clean terminal console (By using ANSI Escape code), current player turn message and player's piece reminder. The formatted game board table are followed.
Invalid input	Print error message in red&bold. New row and column input will be show followed to let player input again.
Player wins	The terminal will be cleaned. Then print the final game board, and win message for the correct player.
Board full, no winner	The terminal will be cleaned. The final game board, “Board Full. No winner.” will be printed.

Example Ouput Screenshots

Player selecting cell on the board	**one player has win the game (Win condition: 5 in a row)**
	

The **primary** output for game is the **formmated game board table**. The player only know how to make the next move by checking the game board. A neat and aesthetically pleasing game board can enhance the gaming experience, so using `tabulate` ensures it is output with proper formatting and spacing.

Reusability and Portability

Reusability means parts of the program can be reused without rewriting everything again. In this project, `board_print`, `check_win_v2`, and `is_board_full` are separated from the main loop to define as a

function so that the same logic can be called from different places. Only the value passes into the function have to change, but different data can be process in the same logic

Portability means the program can run on different machines (The run-time environment have to meet the requirements)

Requirement	Notes
Python 3	Stated in `README.md`
`tabulate`, `termcolor`	Listed in `requirements.txt` & Stated in `README.md`; User have to install them via `pip install / pip3 install`. to run the program.
Terminal with ANSI support	Terminal cleaning and colours assume a typical terminal; Very limited environments may not show colours normally or may cannot recognize the ANSI escape codes.

This program is written in Python 3.14.3, as python have cross-platform capabilities, it allowed it operate on any operating system that have a compatible Python interpreter.

> **IT RUN ON MY MACHINE**:
> The program run tests on macOS Tahoe 26.4 R.C.(25E241) with Python 3.14.3(homebrew)

Discussion & Conclusion

This project implements the Gomoku game by using Python 3, with the primary goal of achieving the intended game flow in a command-line environment. By defining codes that have to use repeatdly into different functions, the main loop can have a simple struction, makeing further development easier. In this project, we can not guarantee that player follow the our expected way to play the game, the data validation part help us solve this problem. By checking non-empty & datatype & range & rules, invalid input can be discarded and let the user to input again until the input is valid. This ensures that the program will not terminate due to errors and that occupied cells will not be replaced. Different libraries is used to enhance the game experience, like `tabulate` use formatted table to enhance the readability of the game board, `termcolor` help better identification and differentiation of information. By counting the number of consecutive pieces in the four directions surrounding the final placement position, the program achieves a more efficient algorithm and reduces the number of traversals.

Future Development Outlook

The project is current are on two-player matches and does not include human-vs-machine gameplay. The interface relies on terminal interface, so color rendering and screen clearing may vary across different environments.

Future project refactoring could be pursued in the following areas:

1. Player vs. Machine (PvM): Utilize an API to request a model to response, let the AI to make decisions by providing the current game board. To avoid issues with API unavailability, it is recommended to allow players to select the preferred match mode (PvP / PvM) before starting, while also include a status check for the API before selecting.
2. Graphical User Interface (GUI): Render the game board as a clickable grid, it can be add more elements like music, animations, and a timer. Standalone window can be provided in GUI, no more

terminal.

Program full code

```
# Gomoku Game
# 25-26 ICT SBA Guided Task 1
# Library required: tabulate, termcolor

# Dependencies
try:
    from tabulate import tabulate  # type: ignore
    from termcolor import colored, cprint  # type: ignore
except ModuleNotFoundError as e:
    raise ModuleNotFoundError(
        "Missing dependency. Please follow instructions from README.md"
    ) from e

# Varaiable
turn = 0
empty_cell_indicator = " "
player_1_indicator = "●"
player_2_indicator = "○"

board = []
num_row_column = 15 # Board Size
for i in range(num_row_column):
    board.append([])
    for r in range(num_row_column):
        board[i].append(empty_cell_indicator)

# Function
## Board Print
def board_print(board):
    headers = [" "]
    for i in range(1, num_row_column + 1):
        headers.append(str(i))

    rows = []
    for i in range(num_row_column):
        row = [str(i + 1)]
        for cell in board[i]:
            row.append(cell)
        rows.append(row)
    print(
        tabulate(
            rows,
            headers=headers,
            tablefmt="rounded_grid",
            stralign="center",
            numalign="center",
        )
```

```
    )
    print()

## Check Win (V2)
def check_win_v2(board, row, col, indicator):
    directions = [
        (0, 1),   # Horizontal
        (1, 0),   # Vertical
        (1, 1),   # Right-down diagonal
        (1, -1)   # Left-down diagonal
    ]
    for dr, dc in directions:
        count = 1  # Initial count

        # Positive
        r, c = row + dr, col + dc
        while 0 <= r < len(board) and 0 <= c < len(board[0]) and board[r]
[c] == indicator:
            count += 1
            r += dr
            c += dc
        # Negative
        r, c = row - dr, col - dc
        while 0 <= r < len(board) and 0 <= c < len(board[0]) and board[r]
[c] == indicator:
            count += 1
            r -= dr
            c -= dc

        # Satisfy win condition: 5 in a row (or more)
        if count >= 5:
            return True

    return False

# Board Full Check
def is_board_full(board):
    for row in board:
        if empty_cell_indicator in row:
            return False
    return True

# Main
while True:
    print("\033c", end="") # Clean console (ANSI Escape Codes)

    # Player turn Noti
    if turn == 0:
        cprint("Player 1 turn \n", "green", attrs=["bold"])
        cprint("Your piece is: " +player_1_indicator, "magenta", attrs=
["bold"])
    else:
        cprint("Player 2 turn \n", "red", attrs=["bold"])
        cprint("Your piece is: " +player_2_indicator, "magenta", attrs=
```

```
["bold"])
    board_print(board)

    # Player input & Data validation
    while True:
        while True:
            selected_row = input(colored("Enter the row: ", "blue", attrs=
["bold"]))
            selected_column = input(colored("Enter the column: ", "blue",
attrs=["bold"]))
            if selected_row == '' or selected_column == '':   # Input
nothing
                cprint("You type nothing!!! Try again, please.", "red",
attrs=["bold"])
                continue    # Jump to next loop: Input again
            selected_row = selected_row.strip() # Remove the whitespace
            selected_column = selected_column.strip()
            if selected_column.isdigit() and selected_row.isdigit():  #
Check the strings only contain digit number
                selected_column = int(selected_column)
                selected_row = int(selected_row)
                break
            else:
                cprint("Invalid input", "red", attrs=["bold"])
                continue

        if selected_row > 0 and selected_row <= num_row_column and
selected_column > 0 and selected_column <= num_row_column:  # Data
validation - within the board size
            if board[selected_row - 1][selected_column - 1] ==
empty_cell_indicator:   # Data validation - EMPTY CELL
                if turn == 0:
                    board[selected_row - 1][selected_column - 1] =
player_1_indicator
                else:
                    board[selected_row - 1][selected_column - 1] =
player_2_indicator
                break  # Exit inner loop - input valid & finish replacing
the cell
            else:
                cprint("This cell is already occupied", "red", attrs=
["bold"]) # Repeat loop - Input valid & Cell occupied
        else:
            cprint("Invalid input", "red", attrs=["bold"]) # Repeat loop -
Input invalid

    # Check win
    if turn == 0:
        if check_win_v2(board, selected_row - 1, selected_column - 1,
player_1_indicator):
            print("\033c", end="")
            board_print(board)
            cprint("Player 1 wins!", "green", attrs=["bold"])
            break    # Exit main loop - Game end with one win
```

```
    else:
        if check_win_v2(board, selected_row - 1, selected_column - 1,
player_2_indicator):
            print("\033c", end="")
            board_print(board)
            cprint("Player 2 wins!", "green", attrs=["bold"])
            break    # Exit main loop - Game end with one win

    # Check the board whether is full or not.
    if is_board_full(board):
        print("\033c", end="")
        board_print(board)
        cprint("Board Full. No winner.", "yellow", attrs=["bold"])
        break    # Exit main loop - Game end with noone win

    turn = 1 - turn # Player round switch
```

References

Library used in this project:

- Tabulate
  - https://github.com/astanin/python-tabulate
- Termcolor
  - https://github.com/termcolor/termcolor

ANSI Escape codes / clean console:

- ANSI Escape Code. (n.d.). Wikipedia. https://en.wikipedia.org/wiki/ANSI_escape_code
- What does printf("\033c") mean? (n.d.). Stack Overflow. https://stackoverflow.com/questions/47503734/what-does-printf-033c-mean

Markdown

- Basic writing and formatting syntax: https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax
- GFM: https://github.github.com/gfm/

- END OF THIS REPORT -

It is not the end of learning, not the end of the DSE, not the end of life, and certainly not the end of suffering.

Thank you!